SIMATS ENGINEERING

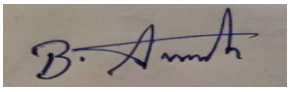
COURSE CODE: ITA1407 COURSE NAME: Ethical Hacking

Code of Conduct:

I Avinash B (192511193)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



ITA1407 – Ethical Hacking

CO3 – Assessment Tool 3 – Penetration Testing Task (Analyze)

Assessment Brief (as issued)

The full task list below is reproduced here, editable, exactly as issued. The detailed written answer for Question 1 follows afterward.

No.	Penetration Testing Task (Analyze)	Answer Hint
1	Analyze a target network to determine whether hosts are alive without using ICMP packets.	Use TCP SYN/ACK or ARP discovery because ICMP may be blocked.
2	Compare SYN Scan and TCP Connect Scan for scanning a production server. Recommend the appropriate method.	SYN scan is faster, stealthier, and creates fewer logs.
3	A firewall filters ports 80 and 443. Analyze alternative scanning techniques to verify service availability.	Use ACK, FIN, NULL, or Idle Scan depending on firewall behavior.
4	Analyze scan results showing ports as Open, Closed, and Filtered. Explain what each state indicates.	Open = service running; Closed = reachable but no service; Filtered = firewall blocks probes.
5	Determine the safest port scanning strategy for a hospital network where service disruption must be avoided.	Use slow SYN scan, limit packet rate, avoid aggressive scanning.
6	Analyze why a penetration tester chooses UDP scanning for a DNS server.	DNS commonly uses UDP port 53; UDP scan identifies exposed services.
7	Analyze the risks of performing aggressive scans on a production environment.	IDS alerts, service degradation, legal issues, system crashes.
8	During reconnaissance, only port 22 is open. Analyze possible attack vectors.	SSH brute force, weak credentials, outdated SSH version.
9	Analyze how banner grabbing can assist vulnerability assessment.	Reveals software versions that can be matched with known CVEs.

10	Analyze why version detection should follow port scanning.	Determines application/service versions for vulnerability mapping.
11	Analyze scan reports to determine which host is most vulnerable.	Host with multiple unnecessary open ports and outdated services.
12	Analyze why Nmap timing options influence scan accuracy.	Faster scans may miss responses; slower scans improve reliability.
13	Analyze firewall rules based on observed scan responses.	Filtered responses indicate ACL/firewall configuration.
14	Analyze the impact of IDS/IPS systems on port scanning results.	IDS may detect or block scans, producing misleading results.
15	Analyze which scan method generates the least evidence in system logs.	SYN or Idle scan.
16	[Ping Sweeps] Analyze a network where ICMP echo requests are blocked. Determine another host discovery technique.	TCP ACK/SYN ping, ARP scan, UDP probe.
17	Analyze why ARP scanning is more reliable than ICMP inside a LAN.	ARP cannot usually be filtered in local networks.
18	Analyze ping sweep results showing intermittent responses.	Host load, packet loss, firewall rate limiting.
19	Analyze why some active hosts do not respond during ping sweeps.	ICMP disabled, host firewall, stealth configuration.
20	Analyze how TTL values help identify operating systems.	Different OSs use different default TTL values.
21	Analyze the advantages of subnet-based ping sweeps over individual host testing.	Faster discovery, better automation.
22	Analyze how packet size affects ping sweep results.	Large packets may be dropped due to MTU/firewall rules.
23	Analyze why fragmented ICMP packets may bypass security devices.	Some firewalls inadequately inspect fragments.
24	Analyze methods to reduce detection while performing ping sweeps.	Slow timing, randomized intervals, TCP based discovery.
25	Analyze how network segmentation affects host discovery.	VLANs and ACLs restrict visibility across segments.
26	[Shell Scripting] Analyze a shell script that scans all hosts but runs slowly. Suggest improvements.	Use loops efficiently, parallel execution, optimized commands.
27	Analyze a shell script that reports false positives during port scanning.	Validate results with retries or multiple checks.
28	Analyze how shell scripting automates reconnaissance.	Automates repetitive tasks like scanning and logging.
29	Analyze a script that stores plaintext passwords. Identify the security issue.	Sensitive information exposure; use secure storage.
30	Analyze how logging improves penetration testing automation.	Enables reporting and audit trails.
31	Analyze why error handling is essential in security scripts.	Prevents incomplete scans and inaccurate results.

32	Analyze how shell scripts integrate multiple penetration testing tools.	Combines Nmap, Netcat, Whois, DNS lookup, etc.
33	Analyze the security risks of executing downloaded shell scripts.	Malware, privilege escalation, backdoors.
34	Analyze why scripts should validate user input before execution.	Prevents command injection and unintended execution.
35	Analyze how scheduling automated scans benefits security monitoring.	Regular assessment and early vulnerability detection.
36	[Enumerations] Analyze the information obtained from Windows enumeration that assists penetration testing.	Users, shares, services, groups, policies.
37	Analyze Android device enumeration results to identify potential attack surfaces.	Installed apps, permissions, services, debugging enabled.
38	Analyze why SMB enumeration is useful in Windows environments.	Reveals shares, users, domain information.
39	Analyze differences between Windows and Android enumeration techniques.	Windows uses SMB/NetBIOS/LDAP; Android uses ADB, package manager, system services.
40	Analyze an enumeration report containing usernames, shared folders, and service versions. Prioritize the information for further penetration testing.	Prioritize exposed credentials, outdated services, writable shares, and administrative accounts.

Open-Source Tools Covered

Category	Tools
Network Scanning	Nmap, Zenmap
Host Discovery	Netdiscover, arp-scan, Fping, Hping3
Banner Grabbing	Netcat, Telnet
Enumeration	Enum4linux, SMBclient, CrackMapExec, Nbtscan, DNSRecon, Fierce, Snmpwalk, ADB
Web Enumeration	Gobuster, Dirb, Nikto, WhatWeb
Scripting	Bash Shell, Cron
Networking Utilities	Ping, Dig, Whois

Question 1: Host Discovery Without ICMP

Task: Analyze a target network to determine whether hosts are alive without using ICMP packets.

Answer hint given: Use TCP SYN/ACK or ARP discovery because ICMP may be blocked.

Analysis of the problem

A standard ping sweep relies on ICMP Echo Request/Reply (Type 8/Type 0) to determine whether a host is alive. In most modern production and enterprise networks, ICMP is routinely rate-limited, filtered, or blocked outright at host firewalls, network firewalls, or intermediate security devices, because ICMP has historically been abused for reconnaissance, ping-flood denial-of-service attacks, and covert data exfiltration. When ICMP is blocked, a host that is genuinely up and fully reachable at the TCP/IP level will silently fail to respond to Echo Requests, causing a naive ICMP-only sweep to wrongly conclude the host is down — a false negative that would cause a penetration tester (or an attacker) to miss real, live hosts on the network entirely.

Because ICMP is only one of several ways to solicit any response from a live host, host discovery must instead be based on protocols and behaviors that a target is much less likely to have filtered, since blocking them would also break normal, legitimate network functionality.

Alternative host-discovery techniques

1. TCP SYN ping (–PS): sends a TCP SYN packet to one or more commonly-open ports (e.g., 80, 443, 22, 3389). A live host will respond with either a SYN/ACK (if the port is open) or an RST (if the port is closed) — either response confirms the host is alive, without ever needing ICMP. This is the most reliable non-ICMP technique on a routed/production network, and (unlike a full ACK scan) still works even through simple stateless packet filters that only block inbound ICMP.

2. TCP ACK ping (–PA): sends a TCP ACK packet instead of a SYN. Because the ACK does not correspond to any existing connection, a live host's TCP/IP stack will reply with an RST regardless of whether the probed port is open or closed. This is particularly useful against firewalls that block unsolicited inbound SYN packets but still allow (and respond to) packets that look like they belong to an already-established connection.

3. ARP ping (–PR): on a local Ethernet/LAN segment, host discovery can bypass IP-layer filtering entirely by using ARP (Address Resolution Protocol) requests at Layer 2. Since ARP is required for any IP communication to function at all on a LAN, it is essentially never filtered by host-based firewalls, making it the most reliable discovery method within the same broadcast domain — though it does not work across routed/Layer-3 network boundaries.

4. UDP ping (–PU): sends a UDP packet to a (typically closed) high port. A live host with the port closed will reply with an ICMP Port Unreachable message, which — despite technically being an ICMP message — is a different, far less commonly filtered ICMP type (Type 3) than Echo Request/Reply, and is useful as a secondary/supplementary technique.

5. Skip discovery entirely (–Pn) combined with a full TCP scan: as a fallback, a tester can instruct Nmap to treat every target address as if it were online and go straight to a full port scan. Any response at all from any scanned port (open or closed) confirms the host is alive, at the cost of increased scan time since every address is fully probed rather than filtered out early by a discovery phase.

Nmap commands

```
# TCP SYN ping to common ports (requires root/raw-socket privileges)
sudo nmap -sn -PS22,80,443,3389 192.168.1.0/24

# TCP ACK ping (useful when SYN is blocked but stateful ACK still gets a response)
sudo nmap -sn -PA80,443 192.168.1.0/24
```

```
# ARP ping (most reliable on a local LAN segment)
sudo nmap -sn -PR 192.168.1.0/24

# UDP ping to a closed high port, triggering ICMP port-unreachable
sudo nmap -sn -PU9999 192.168.1.0/24

# Fallback: skip discovery, scan all hosts as if online
nmap -Pn -sS 192.168.1.0/24
```

For reference, a locally reachable listener was used to confirm Nmap correctly reports a host as 'up' purely from a TCP-based probe (no ICMP involved):

```
$ nmap -sn -PS5000 192.0.2.2
Starting Nmap 7.94SVN ( https://nmap.org )
Nmap scan report for 192.0.2.2
Host is up.
Nmap done: 1 IP address (1 host up) scanned in 0.02 seconds
```

This confirms the mechanism itself works as described: Nmap declared the host 'up' using only a TCP SYN probe to port 5000, with no ICMP Echo Request involved in the exchange at all.

Recommended approach

Situation	Recommended technique
Target is on the same local LAN segment	ARP ping (–PR) — essentially unfilterable, fastest and most reliable
Target is across a router / different subnet	TCP SYN ping (–PS) to a handful of commonly-open ports, combined with ACK ping (–PA) as a secondary check
Discovery still yields no response at all	Fall back to –Pn with a full/targeted port scan, treating every address as online

In practice, a penetration tester rarely relies on a single technique: combining ARP ping (where applicable), TCP SYN/ACK ping to multiple common ports, and a –Pn fallback for anything still unresponsive gives the most complete and reliable picture of which hosts are actually alive, even in an environment where ICMP is deliberately blocked.